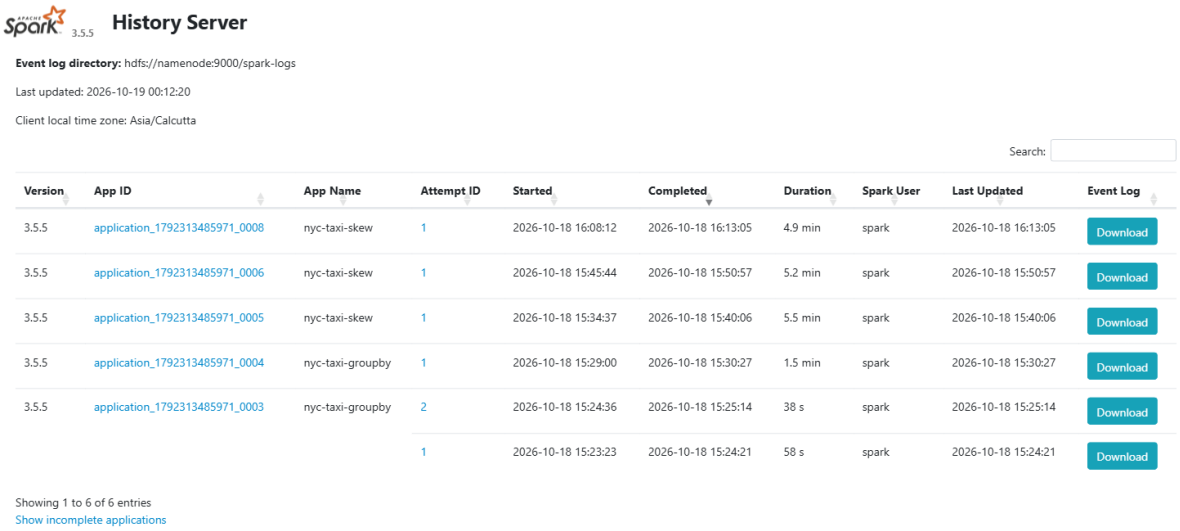
Part 4 — Spark on YARN

I ran the NYC yellow-taxi parquet on YARN in cluster mode. One job is a groupBy. The second job builds a skewed copy of the same rows and then fixes that skew. Both finished on the development queue. The History Server is http://10.1.0.213:18080/. The skew comparison is application_1792313485971_0008. The groupBy that succeeded is application_1792313485971_0004.



History Server

Event log directory: hdfs://namenode:9000/spark-logs

Last updated: 2026-10-19 00:12:20

Client local time zone: Asia/Calcutta

Search:

Version	App ID	App Name	Attempt ID	Started	Completed	Duration	Spark User	Last Updated	Event Log
3.5.5	application_1792313485971_0008	nyc-taxi-skew	1	2026-10-18 16:08:12	2026-10-18 16:13:05	4.9 min	spark	2026-10-18 16:13:05	Download
3.5.5	application_1792313485971_0006	nyc-taxi-skew	1	2026-10-18 15:45:44	2026-10-18 15:50:57	5.2 min	spark	2026-10-18 15:50:57	Download
3.5.5	application_1792313485971_0005	nyc-taxi-skew	1	2026-10-18 15:34:37	2026-10-18 15:40:06	5.5 min	spark	2026-10-18 15:40:06	Download
3.5.5	application_1792313485971_0004	nyc-taxi-groupby	1	2026-10-18 15:29:00	2026-10-18 15:30:27	1.5 min	spark	2026-10-18 15:30:27	Download
3.5.5	application_1792313485971_0003	nyc-taxi-groupby	2	2026-10-18 15:24:36	2026-10-18 15:25:14	38 s	spark	2026-10-18 15:25:14	Download
			1	2026-10-18 15:23:23	2026-10-18 15:24:21	58 s	spark	2026-10-18 15:24:21	Download

Showing 1 to 6 of 6 entries

Show incomplete applications

Figure 1. History Server. nyc-taxi-skew is application_1792313485971_0008 (4.9 min). nyc-taxi-groupby is application_1792313485971_0004 (1.5 min). application_1792313485971_0003 is the earlier groupBy attempt that failed on snap-test.txt.

Submit command and configuration

I submitted from the spark-client pod with spark-submit, master yarn, deploy mode cluster, queue development. The NodeManager image has Java 8 and no Python 3.8, and Spark 3.5 needs Python 3.8 or newer, so the job ships a CPython 3.10 archive from HDFS.

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --name nyc-taxi-groupby \
  --queue development \
  --executor-memory 1g \
  --executor-cores 1 \
  --driver-memory 512m \
  --archives hdfs://namenode:9000/spark/python-3.10.tgz#pyenv \
  --conf spark.pyspark.python=./pyenv/python/bin/python3 \
  --conf spark.pyspark.driver.python=./pyenv/python/bin/python3 \
  --conf spark.executorEnv.PYSPARK_PYTHON=./pyenv/python/bin/python3 \
  --conf spark.yarn.appMasterEnv.PYSPARK_PYTHON=./pyenv/python/bin/python3 \
  --conf spark.executor.memoryOverhead=256m \
  --conf spark.driver.memoryOverhead=256m \
  --conf spark.dynamicAllocation.enabled=true \
  --conf spark.dynamicAllocation.shuffleTracking.enabled=true \
```

```
  --conf spark.dynamicAllocation.minExecutors=1 \
  --conf spark.dynamicAllocation.initialExecutors=1 \
  --conf spark.dynamicAllocation.maxExecutors=2 \
  --conf spark.sql.shuffle.partitions=8 \
  --conf spark.eventLog.enabled=true \
  --conf spark.eventLog.dir=hdfs://namenode:9000/spark-logs \
  --conf spark.yarn.stagingDir=hdfs://namenode:9000/spark/staging \
  /tmp/part4/taxi_groupby.py
```

Dynamic allocation is on, with shuffle tracking, so it does not need an external shuffle service. The cluster only has two NodeManagers, so maxExecutors is 2. Executor memory is 1 GB and driver memory is 512 MB. Each container also gets 256 MB of overhead, which is why the ApplicationMaster shows up as 768 MB. The skew job uses the same flags with --name nyc-taxi-skew and taxi_skew.py. The scripts are on the VM at /opt/hadoop-lab/spark/jobs/.

The Environment tab on the History Server lists every Spark property. I left the rows from this submit command visible so they fit on one page. Executor memory is 1g, driver memory is 512m, dynamic allocation is true, the queue is development, and the deploy mode is cluster.

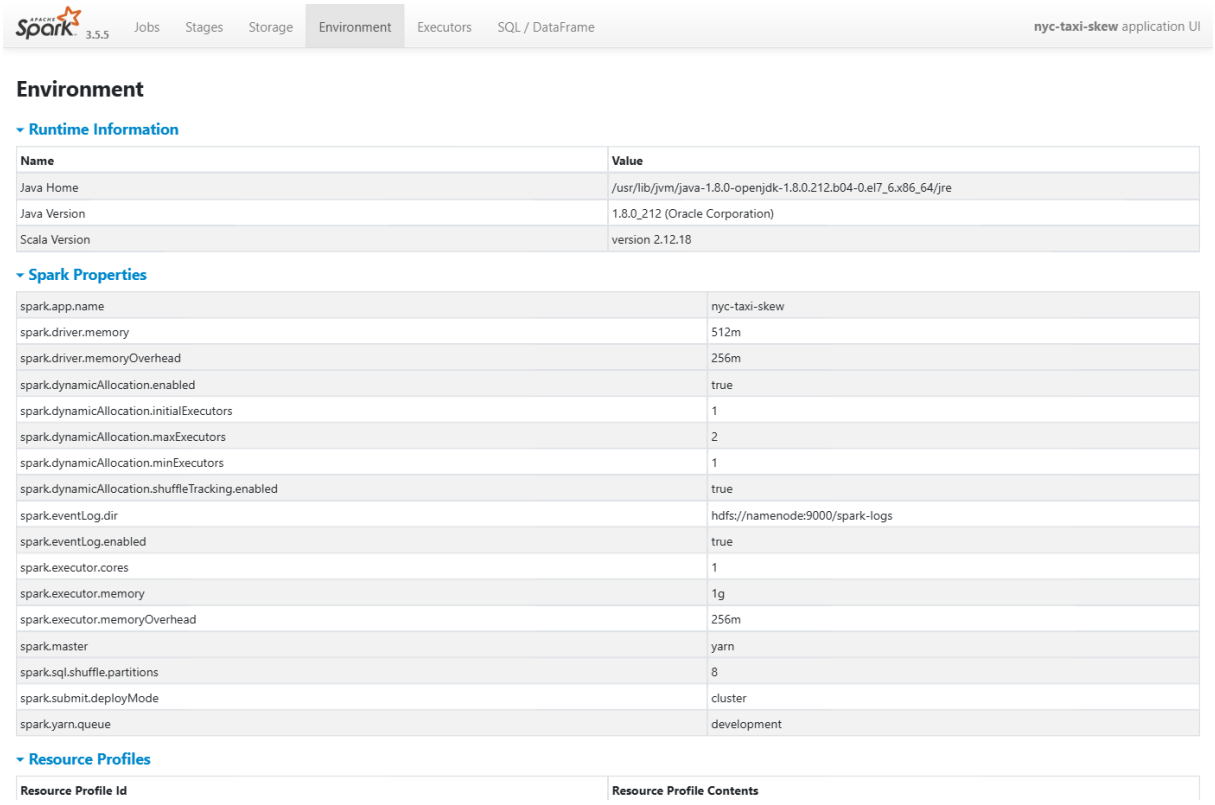


Figure 2. Environment tab for application_1792313485971_0008. spark.executor.memory is 1g, spark.driver.memory is 512m, dynamic allocation is on, maxExecutors is 2, and the queue is development.

The read path is hdfs://namenode:9000/datasets/nyc-taxi/*.parquet. snap-test.txt sits in that directory and is not parquet. The first attempt, application_1792313485971_0003, failed on that file. I left the file in place and pointed the job at the parquet files only.

GroupBy

The job reads the February through June 2019 yellow-trip parquet, groups by PULocationID and payment_type, and writes trip count, total fare, and average distance. The output is parquet at hdfs://namenode:9000/output/nyc-taxi/groupby.

It finished SUCCEEDED on the development queue. 1,288 rows, 59.9 seconds. On the stages page, stage 1 is the read: 5 tasks, 136.8 MiB in, 24 seconds. Stage 8 writes the result, 22.7 KiB out. The submitted times on this page are UTC. 10:00 UTC is 15:30 IST, which matches the History Server list.

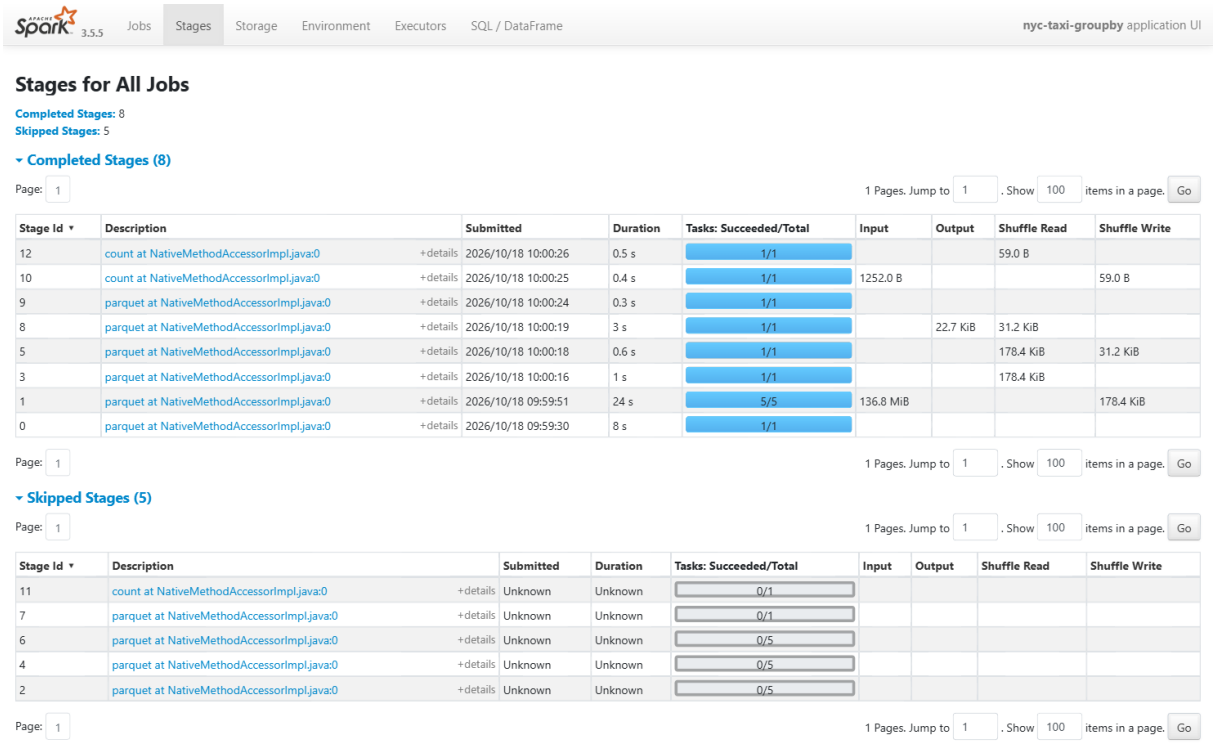


Figure 3. Stages for nyc-taxi-groupby, application_1792313485971_0004. Eight stages completed. Stage 1 read 136.8 MiB across 5 tasks.

How I created the skew

The real pickup zones are spread out, so a normal groupBy does not show one hot task. After the read I replaced the pickup key: if a random draw is under 0.90 the row becomes pickup key 1, otherwise it keeps its real PULocationID. That is the whole skew. Nothing else in the file is changed.

The job printed SKEW total_rows=36961944 hot_key=1 hot_rows=33263682 hot_pct=90.0. About 33.3 million of 37.0 million rows sit on one key. A hash shuffle puts that key in a single task, so one task does almost all of the work.

Salt and repartition

The fix is in the same application, application_1792313485971_0008 (nyc-taxi-skew, 4.9 minutes). I salted only the hot key. Key 1 gets a random bucket from 0 to 31. Every other zone gets a stable bucket from the

zone id. Then I repartition on that salt, aggregate the partial totals, and add them back up by pickup key. AQE is off for this comparison so the difference is the salt and the repartition.

Both sides write 260 groups, so the rollup matches. The before output is hdfs://namenode:9000/output/nyc-taxi/skew-before and the after output is skew-after.

Stage 22 is the plain groupBy. Stage 15 is the salted repartition. These are the numbers from the History Server pages below.

What I measured	Before the salt, stage 22	After the salt, stage 15
Tasks	8	32
Stage duration	40 s	12 s
Time added across the tasks	43 s	22 s
Slowest task	37 s	2 s
Median task	0.6 s	0.6 s
Records in the slowest task	33,726,044	3,618,285
Shuffle read for the stage	17.2 MiB / 36,962,712 records	425.3 MiB / 36,961,944 records
Spill	768 MiB memory, 4.8 MiB disk	No spill on this stage
The action on the clock	15.3 s	39.7 s

The salted stage moves more bytes, 425.3 MiB against 17.2 MiB, because it shuffles the full rows out into 32 buckets. That is why the action on the clock went from 15.3 seconds to 39.7 seconds. The stage page is the comparison that shows the skew is gone: the slowest task drops from 37 seconds and 33.7 million records to 2 seconds and 3.6 million records, and the 768 MiB spill is gone.

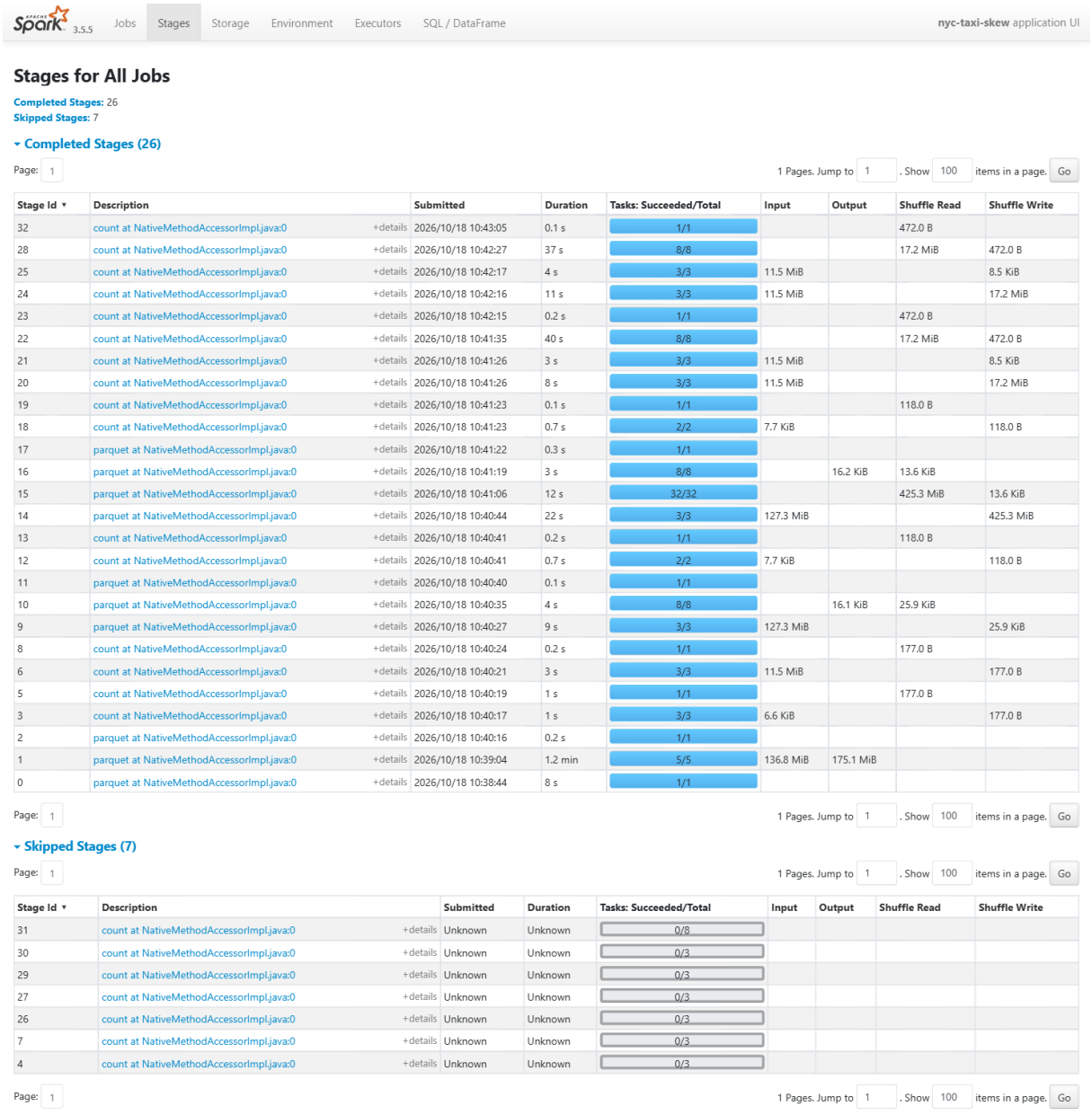


Figure 4. Stages for nyc-taxi-skew. Stage 22 is the skewed groupBy: 40 s, 8/8 tasks, shuffle read 17.2 MiB. Stage 15 is the salted repartition: 12 s, 32/32 tasks, shuffle read 425.3 MiB. Stage 28, still 37 s and 8 tasks, is the later join. Submitted times here are UTC.

Opening stage 22 shows where the 40 seconds went. Eight tasks finished. Seven of them took 2 seconds or less. Task index 3 ran for 37 seconds, read 7.1 MiB / 33,726,044 records, and spilled 768 MiB of memory and 4.8 MiB of disk. The median task was 0.6 seconds.

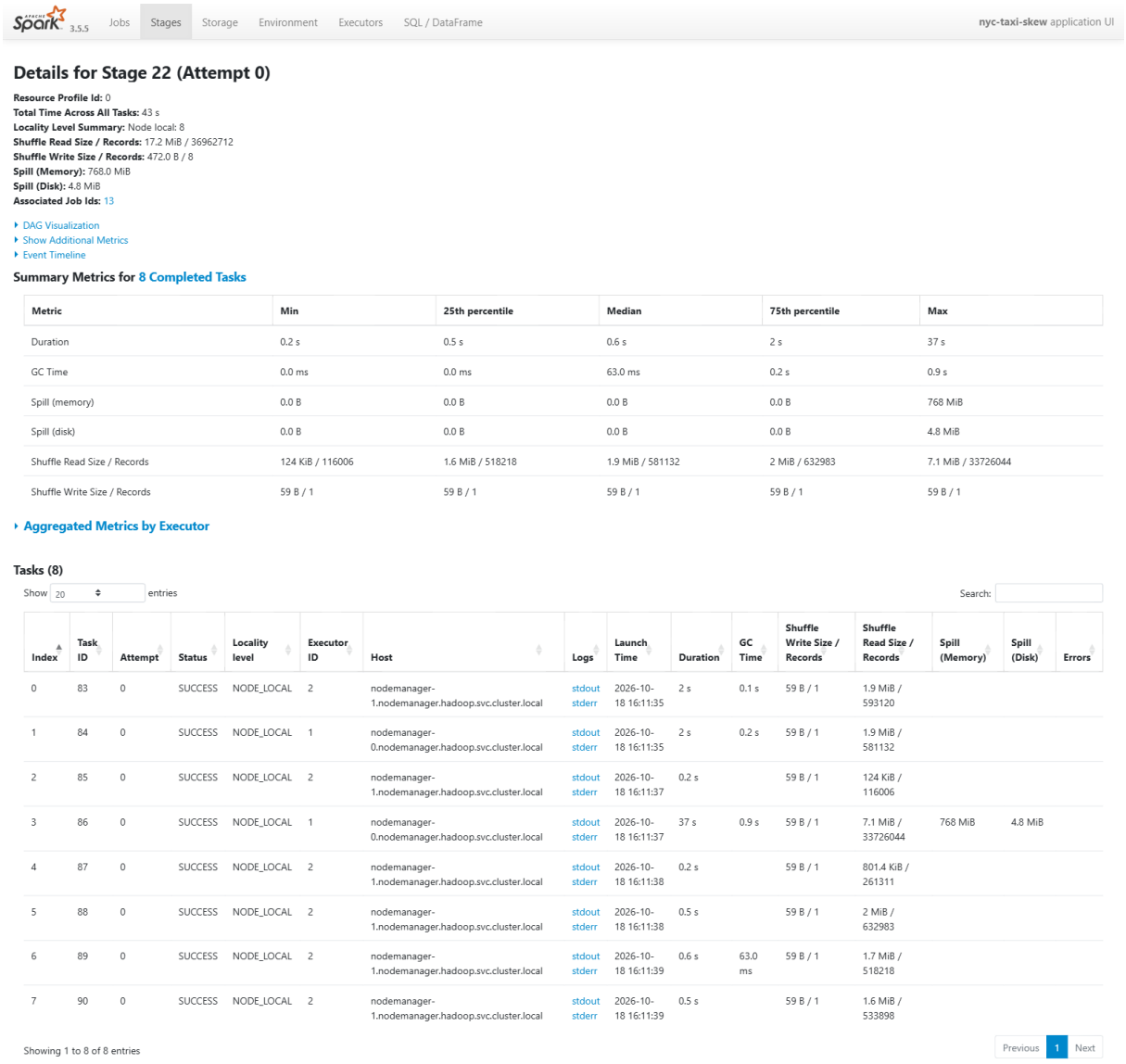


Figure 5. Stage 22 before the salt. Duration max is 37 s, median is 0.6 s. Task 3 read 33,726,044 records and spilled 768 MiB. The other seven tasks stayed under 2 seconds.

Stage 15 is the same rows after the salt. Thirty-two tasks, total time across them 22 seconds. The slowest task on the summary is 2 seconds. The largest shuffle read is 43.7 MiB / 3,618,285 records. There is no spill line on this stage. Shuffle write is 13.6 KiB / 277 partial rows, which is the rollup input.

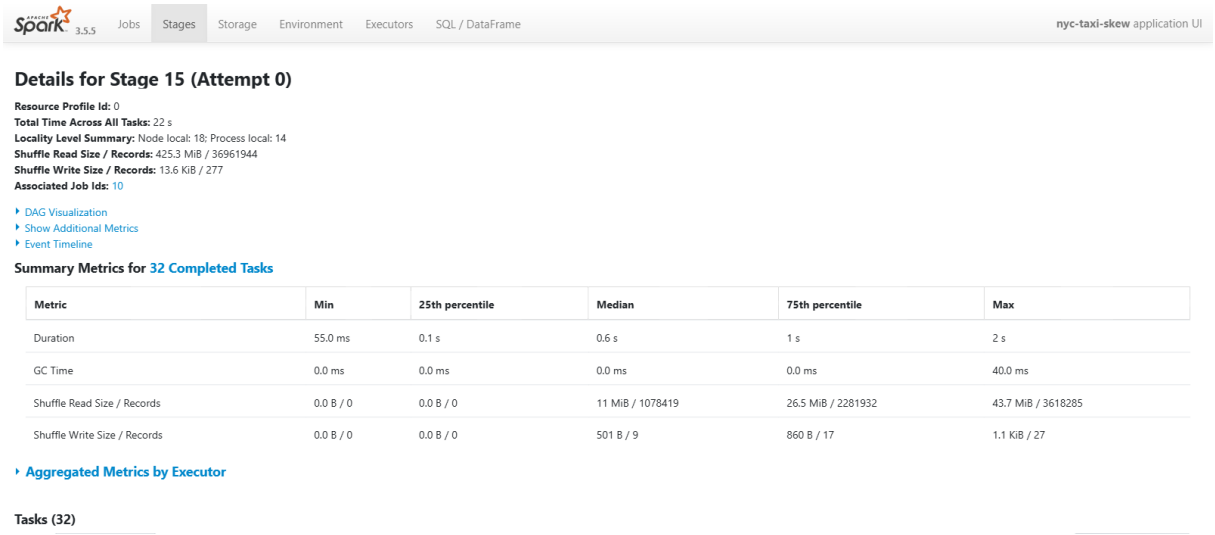
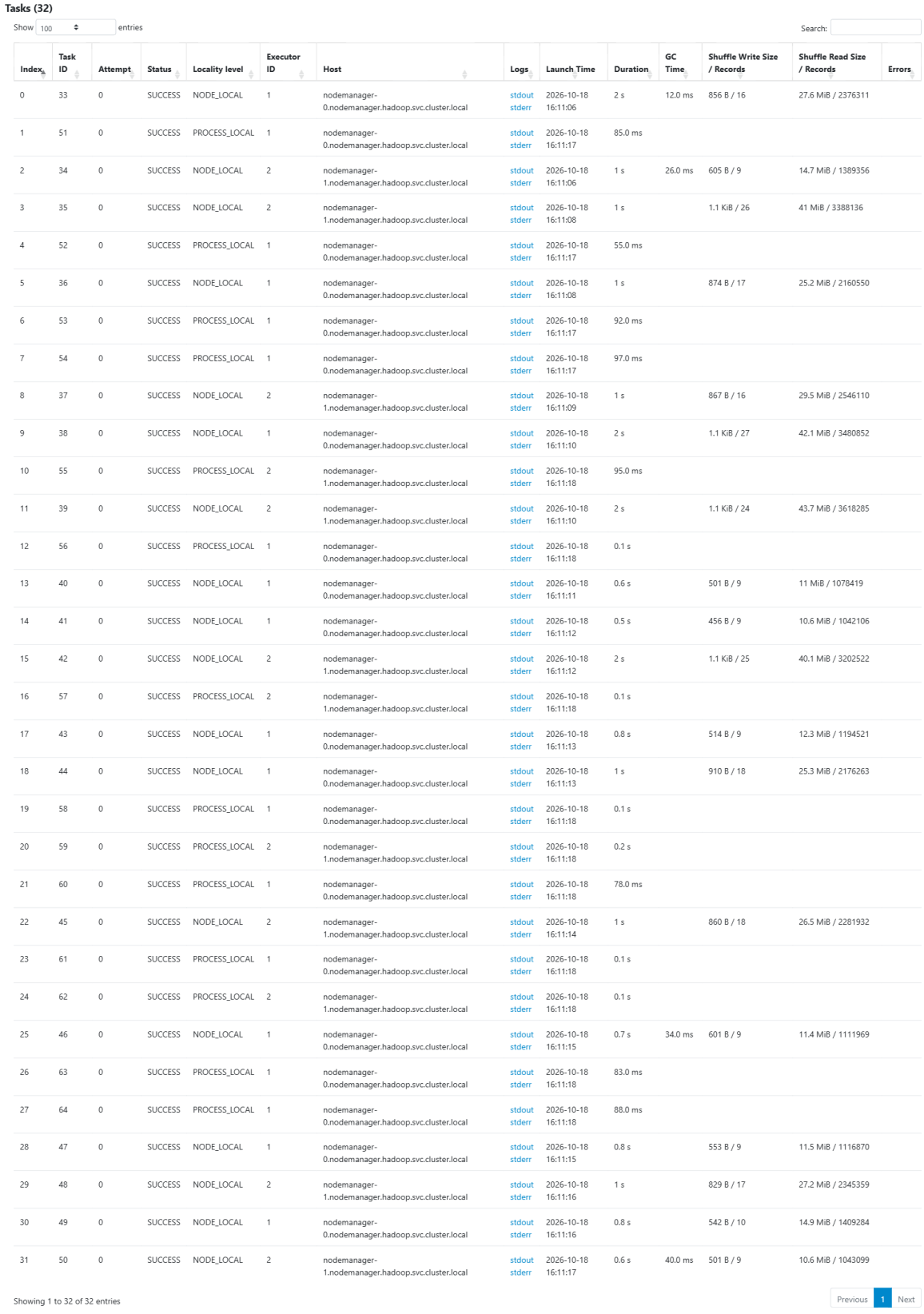


Figure 6. Stage 15 after the salt. 32 tasks, total time 22 s, duration max 2 s. Shuffle read is 425.3 MiB spread across the tasks. The max task holds 3,618,285 records.



Tasks (32)

Show 100 entries — Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	33	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:06	2 s	12.0 ms	856 B / 16	27.6 MiB / 2376311	
1	51	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:17	85.0 ms				
2	34	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:06	1 s	26.0 ms	605 B / 9	14.7 MiB / 1389356	
3	35	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:08	1 s		1.1 KiB / 26	41 MiB / 3388136	
4	52	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:17	55.0 ms				
5	36	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:08	1 s		874 B / 17	25.2 MiB / 2160550	
6	53	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:17	92.0 ms				
7	54	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:17	97.0 ms				
8	37	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:09	1 s		867 B / 16	29.5 MiB / 2546110	
9	38	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:10	2 s		1.1 KiB / 27	42.1 MiB / 3480852	
10	55	0	SUCCESS	PROCESS_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	95.0 ms				
11	39	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:10	2 s		1.1 KiB / 24	43.7 MiB / 3618285	
12	56	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	0.1 s				
13	40	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:11	0.6 s		501 B / 9	11 MiB / 1078419	
14	41	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:12	0.5 s		456 B / 9	10.6 MiB / 1042106	
15	42	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:12	2 s		1.1 KiB / 25	40.1 MiB / 3202522	
16	57	0	SUCCESS	PROCESS_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	0.1 s				
17	43	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:13	0.8 s		514 B / 9	12.3 MiB / 1194521	
18	44	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:13	1 s		910 B / 18	25.3 MiB / 2176263	
19	58	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	0.1 s				
20	59	0	SUCCESS	PROCESS_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	0.2 s				
21	60	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	78.0 ms				
22	45	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:14	1 s		860 B / 18	26.5 MiB / 2281932	
23	61	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	0.1 s				
24	62	0	SUCCESS	PROCESS_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	0.1 s				
25	46	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:15	0.7 s	34.0 ms	601 B / 9	11.4 MiB / 1111969	
26	63	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	83.0 ms				
27	64	0	SUCCESS	PROCESS_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:18	88.0 ms				
28	47	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:15	0.8 s		553 B / 9	11.5 MiB / 1116870	
29	48	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:16	1 s		829 B / 17	27.2 MiB / 2345359	
30	49	0	SUCCESS	NODE_LOCAL	1	nodemanager-0.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:16	0.8 s		542 B / 10	14.9 MiB / 1409284	
31	50	0	SUCCESS	NODE_LOCAL	2	nodemanager-1.nodemanager.hadoop.svc.cluster.local	stdout stderr	2026-10-18 16:11:17	0.6 s	40.0 ms	501 B / 9	10.6 MiB / 1043099	

Showing 1 to 32 of 32 entries — Previous 1 Next

Figure 7. All 32 tasks on stage 15. Durations run from 55 ms to 2 s. No task is holding the 33 million-row key, and none of them spill.

AQE

The same job also joins the skewed rows to a small zone table. Broadcast is turned off (spark.sql.autoBroadcastJoinThreshold=-1) so the join has to shuffle. The first join runs with AQE off. The second turns AQE on, turns coalesce off, and sets the skew split threshold to 256 KB and the split target to 512 KB.

The join times were 51.5 seconds with AQE off and 49.6 seconds with AQE on. The compressed hot partition is only about 7 MB, so the split target never asked for more than one task. On the stages list, stage 28 is that join: 37 seconds, 8 tasks, shuffle read 17.2 MiB, the same shape as stage 22. AQE left the hot key on one task. The fix that shows up on the stage page is the salt and the repartition, stage 22 against stage 15.

Where to look

History Server list: http://10.1.0.213:18080/

GroupBy stages: http://10.1.0.213:18080/history/application_1792313485971_0004/1/stages/

Skew stages: http://10.1.0.213:18080/history/application_1792313485971_0008/1/stages/

Hot task, before the salt:
http://10.1.0.213:18080/history/application_1792313485971_0008/1/stages/stage/?id=22&attempt=0

After the salt:
http://10.1.0.213:18080/history/application_1792313485971_0008/1/stages/stage/?id=15&attempt=0

ResourceManager for the same apps: http://10.1.0.213:8088/. Both ran on the development queue.